NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming ([Course code])

Rùa Toán: A Mathematics Practice Website

A source-attributed problem library for lower secondary students

Group: [Group name/number]
Class: [Class name]
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/thunopro/ruatoan
Demo: https://ruatoan.vercel.app

Team Members

No.	Full name	Student ID	Role
1	[Full name]	[Student ID]	[Role and work]
2	[Full name]	[Student ID]	[Role and work]
3	[Full name]	[Student ID]	[Role and work]
4	[Full name]	[Student ID]	[Role and work]

Hanoi, November 2026

Declaration

We declare that this report describes the Rùa Toán website developed for Web Design and Programming. The group must verify this statement, complete the member and contribution information, and cite any external material or AI assistance used in the submitted version.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Member 1	[Describe verified contribution]	[%]
Member 2	[Describe verified contribution]	[%]
Member 3	[Describe verified contribution]	[%]
Member 4	[Describe verified contribution]	[%]
	Total	**[100]**

Contribution percentages are pending confirmation from the group and must sum to 100%.

Table of Contents

1 Introduction . . . 5
1.1 Background and motivation . . . 5
1.2 Project objectives . . . 5
1.3 Scope and boundaries . . . 5
1.4 Technology overview . . . 6
1.5 Catalogue size . . . 6
1.6 Report evidence and snapshot . . . 6
2 Analysis and Design . . . 6
2.1 Requirements . . . 6
2.1.1 Functional requirements . . . 7
2.1.2 Non-functional requirements and constraints . . . 7
2.2 Interface design . . . 8
2.3 System architecture . . . 15
2.4 Data design . . . 15
2.5 Database design . . . 17
2.6 API design . . . 18
3 Implementation . . . 19
3.1 Project structure and runtime responsibilities . . . 19
3.2 Content loading, indexing, and search . . . 19
3.3 Statement and mathematics rendering . . . 20
3.4 Page composition and interactions . . . 20
3.5 Content preparation pipeline . . . 20
3.6 Google authentication in the current checkout . . . 21
3.7 Database operations and photo API . . . 21
3.8 Deployment and environment . . . 22
4 Results and Conclusion . . . 22
4.1 Evidence collected . . . 22
4.2 What the implementation achieves . . . 23
4.3 Limitations and risks . . . 23
4.4 Recommended next steps . . . 23
4.5 Conclusion . . . 24
References . . . 25

A Appendix 25
A.1 Local setup 25
A.2 Route inventory 25
A.3 Content inventory snapshot 26
A.4 Data pipeline diagram 26
A.5 Glossary 26
A.6 AI assistance and academic disclosure 26

List of Figures

1 Public demo home page: primary actions, catalogue totals, daily problem, topics, and recent collections. . . . 9
2 Practice page at desktop width, showing topic navigation and problem cards. . . . 10
3 Problem page showing mathematical text, source context, submission control, and related navigation. 11
4 Exam collection page with ordered statements and print-oriented content. . . . 12
5 Topic landing page grouping subject-specific lessons and materials. . . . 13
6 Google sign-in page visible in the public demo snapshot. . . . 13
7 Home page captured at a 390-pixel viewport width. . . . 14
8 Logical system architecture. Content, photo persistence, and sign-in use separate data paths. . . . 15
9 Catalogue and user-related records by storage location. Photo identity and OAuth session remain separate. . . . 16
10 Collection counts by source kind in the inspected JSON corpus. . . . 16
11 Collection counts by target grade; grade 10 denotes specialised entrance material. . . . 17
12 Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars. . . . 17
13 Photo API flow from client-side preparation through database storage and device-scoped deletion. 18
14 Content preparation and runtime consumption. Source scripts are optional offline tools; they are not called during page requests. . . . 26

List of Tables

1 Technologies identified in the inspected project files. . . . 6
2 Catalogue groups counted directly from non-sample JSON records. . . . 15
3 Implemented route responsibilities and the explicit response behaviour visible in source. . . . 18
4 Source preparation responsibilities visible in the scripts and content layout. . . . 21
5 Evidence level for the claims made in this report. . . . 22
6 Application page and API route map. . . . 25

1. Introduction

1.1. Background and motivation

Students preparing for lower-secondary mathematics competitions often need to consult material scattered across exam collections, topic exercises, magazines, books, and international olympiad archives. Rùa Toán brings a large, source-attributed catalogue of these materials into one Vietnamese-language website. Its core use case is to help a learner find a problem, read its statement with correctly rendered notation, work it out on paper, and keep a photograph of the attempt.

The product is intended for students in grades 6–9, including learners preparing for gifted student examinations and entrance examinations for grade 10 specialised schools. Teachers and parents are secondary users who can browse complete collections and follow links to the original sources. The public catalogue includes five topic areas: number theory, algebra, inequalities, geometry, and combinatorics. It also includes magazines, topic collections, and selected olympiad papers.

1.2. Project objectives

The project aims to:

O1. consolidate a large mathematics catalogue while keeping collection, grade, topic, year, province, and source context attached to each problem;

O2. provide practical browsing tools for locating a collection or an individual problem;

O3. preserve mathematical notation and source attribution in both problem and print views;

O4. let learners keep a browser-local progress summary and submit photographs of written work without making those photographs part of the static content catalogue; and

O5. offer reference lessons, books, magazines, and documents through topic pages and outbound links.

The current source tree also contains a Google OAuth sign-in and sign-up interface. It uses Auth.js with JWT sessions and displays the signed-in name or email in the site header. The photo-submission API remains based on a separate browser device token; the report does not assume that OAuth identity is linked to uploaded photographs.

1.3. Scope and boundaries

The implemented product includes a daily problem, searchable and filterable problem lists, exam collections, topic pages and lessons, books and document listings, mathematical text rendering, printing, browser-local progress state, and photo submission and management. Google OAuth pages and session-aware navigation are present in the current checkout.

The application does not provide automatic answer marking, a teacher administration panel, cross-device synchronisation for photo progress, or a database-backed application user profile. Authentication source and deployment configuration may also differ: the OAuth files in the inspected working tree are uncommitted, so their presence in the live deployment has not been established. The database-backed photo feature and OAuth session are separate mechanisms in the code.

1.4. Technology overview

The project stack and each component's responsibility are summarised in Table 1.

Layer	Technology	Use in Rùa Toán
Web framework	Next.js 16, App Router	File-system routes, server-rendered pages, metadata, and route handlers
UI	React 19, TypeScript, CSS	Reusable page components and browser interactions
Mathematics	KaTeX 0.16	Server-side rendering of LaTeX in problem and lesson content
Catalogue	JSON files	Versioned collections, problems, lessons, books, and documents
Submission data	PostgreSQL and `postgres` client	Photo bytes and metadata in schema `thcs_hard`
Sign-in	Auth.js v5 beta, Google OAuth	Google sign-in and JWT-based session handling in the current checkout
Content tooling	Python and Node.js scripts	Crawl, prepare, transform, and inspect source material
Hosting configuration	Vercel	Next.js deployment configuration; region `sin1` is declared

Table 1. Technologies identified in the inspected project files.

1.5. Catalogue size

The inspected content files contain 3,381 collection records and 33,929 nested problem records, together with 90 lessons, 216 book, magazine, or resource records, and 89 document records. This is 37,705 raw records when each collection, problem, lesson, book, and document entry is counted once. The JSON files occupy approximately 16.6 MB. The figures describe the repository snapshot, not independently verified counts of unique, accurate, or educationally reviewed items. A problem may have more than one topic tag, so topic counts are assignments and do not add up to the number of distinct problems.

1.6. Report evidence and snapshot

This report is based on a source-code and JSON audit of the local Rùa Toán checkout on 1 November 2026. The working tree contains uncommitted changes, including the Auth.js pages; their production deployment is not confirmed. Interface screenshots in Chapter 2 are observations of the public demo on the same date and may not exactly match the current local source. No application build, automated test suite, upload test, or performance benchmark was run for this report. Repository counts and code behaviour are described separately from those unmeasured outcomes.

Section 2 presents the requirements, interface, architecture, data design, and API. Section 3 explains the runtime implementation, content preparation, authentication, and deployment configuration. Section 4 summarises observed results, limitations, and future work. The appendix records setup information, data inventory, and implementation notes.

2. Analysis and Design

2.1. Requirements

Requirements below describe visible behaviour and quality goals represented in the inspected source. They are not claims that every case has passed a formal acceptance test.

2.1.1. Functional requirements

FR1. Browse exam collections by kind; filter collections by grade, year, province, and official-versus-practice classification; search by text and move between result pages.

FR2. Browse individual problems by topic, grade, collection kind, year, or text query; open a problem with its collection context and source attribution.

FR3. Read problem and lesson statements containing inline or display mathematical notation, and navigate to adjacent problems in the same collection.

FR4. Open a collection in its original problem order, move to its problem pages, and print the collection using the print control.

FR5. Browse the five topic areas, lessons, reference documents, books, magazines, and learning resources, following external source links where supplied.

FR6. Submit up to six compressed photographs in one request; view a device's photo history, open an image by UUID, and delete an image when the device token matches.

FR7. Keep a compact progress and submission summary in browser storage, independently of the database image records.

FR8. In the current checkout, offer Google sign-in/sign-up pages, display session identity in the shared header, and let a signed-in user sign out.

2.1.2. Non-functional requirements and constraints

- **Responsive presentation:** pages should remain readable on narrow and wide screens. The CSS contains responsive layout rules; a formal device test has not been recorded.
- **Readable mathematics:** problem text is rendered with KaTeX and has a print-specific exam view. Print output has not been formally measured.
- **Source transparency:** problem and collection pages show source names and link to source pages when URLs are available.
- **Keyboard and assistive-technology support:** the layout includes a skip link, visible keyboard focus, labelled navigation/controls, and reduced-motion CSS. These are implementation details, not evidence of a completed accessibility audit.
- **Bounded image input:** the API accepts WebP/JPEG only, no more than six files per request, 150 KB per file, and 60 images per device in a rolling 24-hour window. These application checks are not a general network-level request-size control.
- **Pseudonymous photo ownership:** a random browser token is hashed before it is stored with a photo. This does not verify a student's identity and is independent of the Google OAuth session.
- **Secret handling:** database and OAuth secrets are supplied through environment variables and must not be committed or printed in the report.
- **Content quality:** records preserve source context, but the project has no measured editorial audit that proves every transcription, link, tag, or year is correct.

2.2. Interface design

The interface presents a dark green, brick-and-hexagon visual background with bright lime controls, cream problem cards, five topic colours, and a turtle mascot. A shared header links to practice, exams, topics, books, and submitted work. The current source adds a Google sign-in control when there is no session and the user's display name/email plus sign-out control when a session is present.

The home page is the discovery dashboard: it shows the main actions, live catalogue totals, a date-selected problem, topic totals, and recently ordered collections. The practice page uses topic tabs, query parameters for filters, twenty results per page, problem cards, and a pager. The exam list provides collection filters and twenty-four collections per page. A problem page keeps its source visible and groups the statement, photo submission, previous and next links, and related problems. A collection page preserves question order and offers both a print action and links into individual problems. Topic pages combine lesson, document, book, and exercise counts in one subject-oriented entry point.

Figures 1–6 are screenshots captured from the public demo on 1 November 2026. They document the deployed visual state at that time; they are not unit-test evidence and may differ from uncommitted UI changes in the inspected checkout. The practice, problem, exam, and topic views are shown individually in Figures 2, 3, 4, and 5.

The layout supports narrow widths in its CSS. Figure 7 is a narrow viewport capture of the public demo; it shows layout behaviour but does not replace a documented accessibility or device-compatibility assessment.

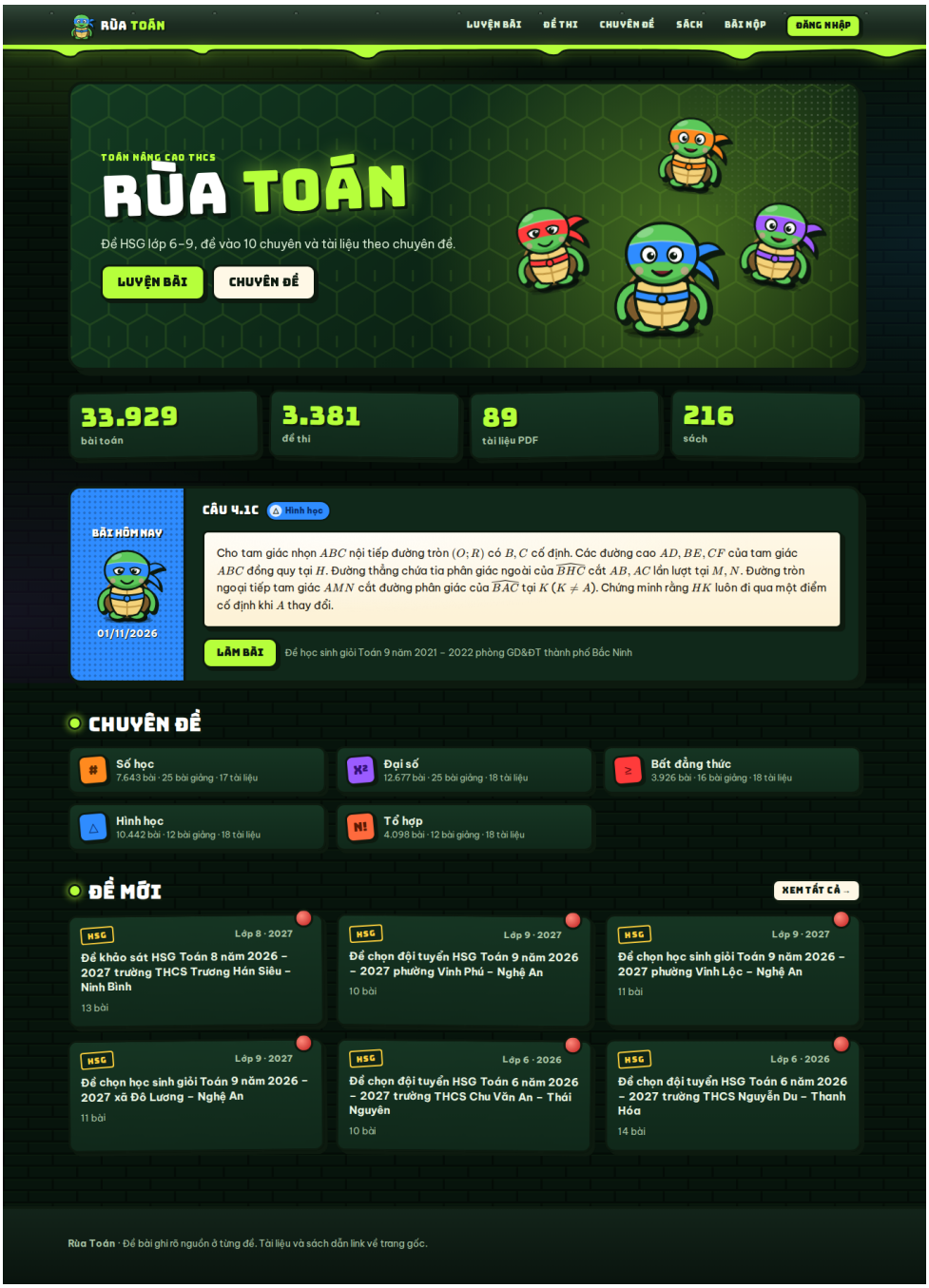


Figure 1. Public demo home page: primary actions, catalogue totals, daily problem, topics, and recent collections.



Figure 2. Practice page at desktop width, showing topic navigation and problem cards.

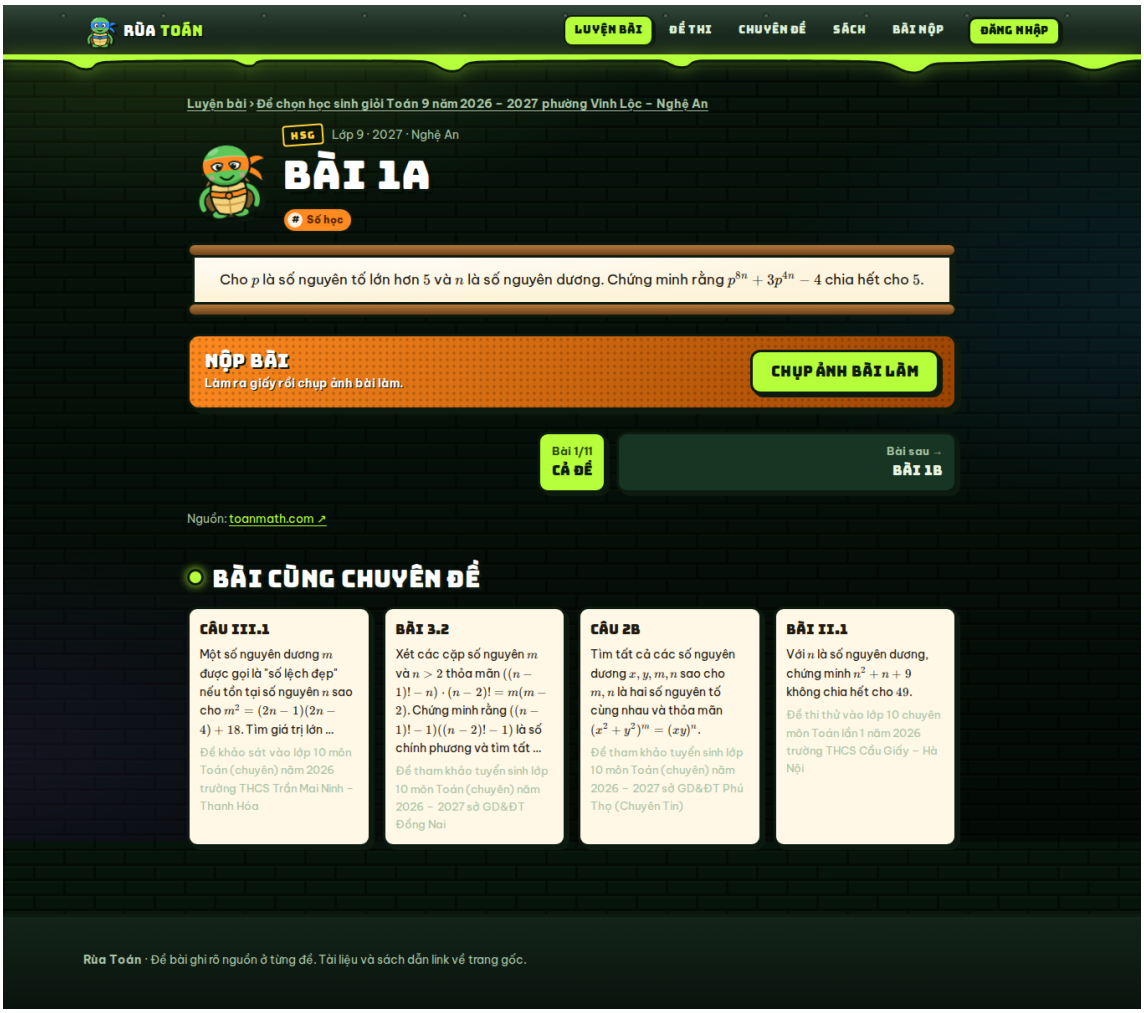


Figure 3. Problem page showing mathematical text, source context, submission control, and related navigation.



Figure 4. Exam collection page with ordered statements and print-oriented content.



Figure 5. Topic landing page grouping subject-specific lessons and materials.

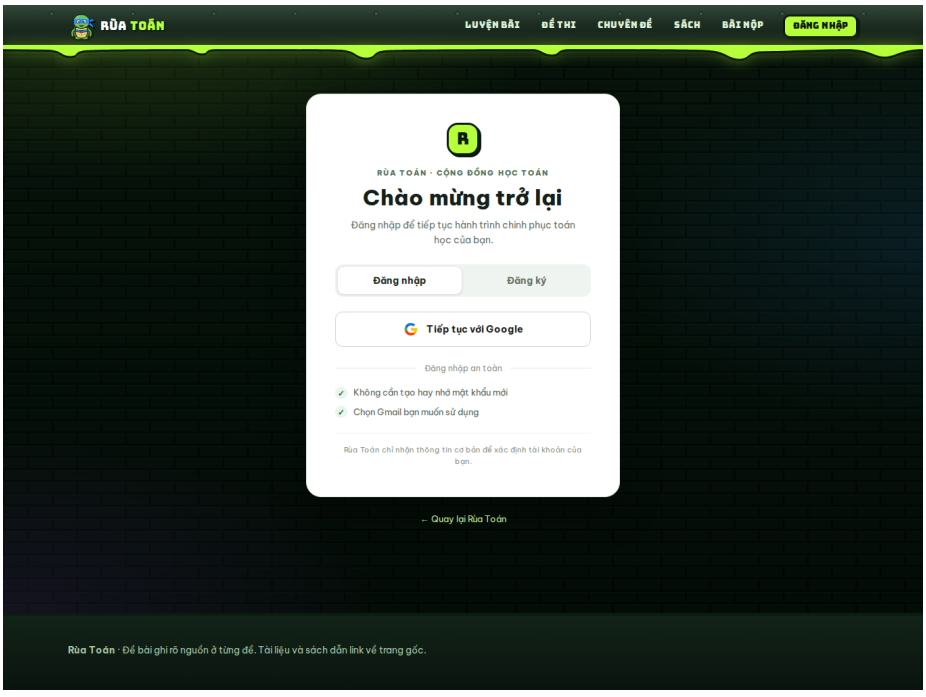


Figure 6. Google sign-in page visible in the public demo snapshot.



Figure 7. Home page captured at a 390-pixel viewport width.

2.3. System architecture

Rùa Toán uses the Next.js App Router [4]. Figure 8 summarises the system boundary. Page and layout files render catalogue pages on the server; client components are used where browser state, event handlers, local storage, or image compression are needed. Static catalogue JSON is loaded by the server-side content module. A separate route-handler API handles photo records in PostgreSQL. Auth.js provides the Google OAuth flow and JWT session in the current checkout. The OAuth session and device token are distinct and are not joined to one another by the photo API.

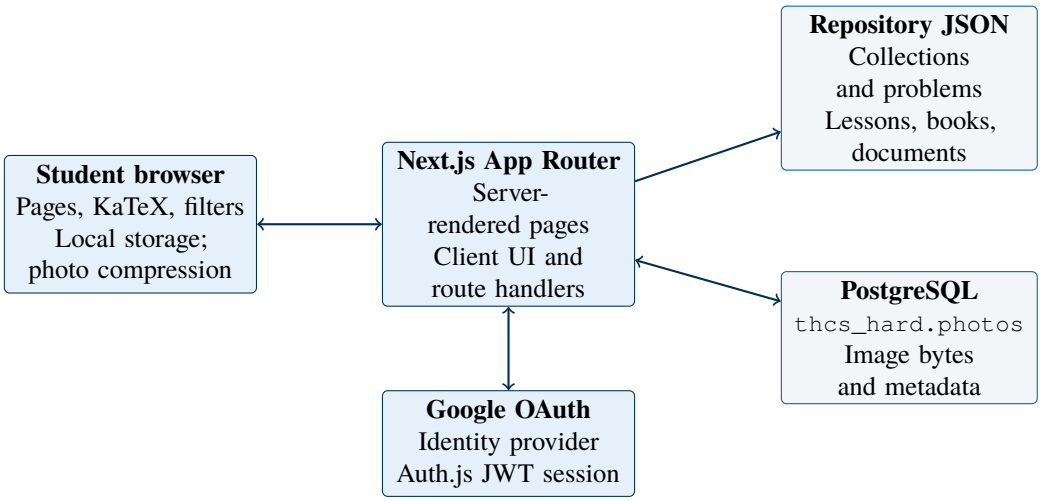


Figure 8. Logical system architecture. Content, photo persistence, and sign-in use separate data paths.

The home page declares a one-hour revalidation interval. Problem and collection detail pages declare a one-day revalidation interval. These declarations describe Next.js route behaviour; no response-time benchmark was collected. The content module also caches parsed file data using file modification times and rebuilds in-memory maps when its signature changes.

2.4. Data design

The catalogue is organised as structured JSON, not as one relational database. The groups and primary fields are summarised in Table 2. There are eleven non-sample collection files under `content/sources/`, seven non-sample lesson files under `content/theory/`, and top-level files `books.json` and `docs.json`. In this repository snapshot, these JSON files occupy about 16.6 MB. Files beginning with an underscore are samples or work-in-progress and are not used when real content exists.

Record group	Count	Primary fields / use
Collections	3,381	ID, kind, title, grade, optional year/province, source name/URL, ordered problems
Problems	33,929	ID, label, statement text, topic tags; nested in a collection
Lessons	90	ID, topic, title, summary, body, optional examples
Books and resources	216	ID, title, author/publisher, grades/topics, description, URL, kind and source
Documents	89	ID, topic, title, author, grade/page metadata, description and source links

Table 2. Catalogue groups counted directly from non-sample JSON records.

Figure 9 shows how the catalogue, browser state, photo rows, and OAuth session relate. The main nested relationship is *Collection contains an ordered array of Problems*. Collection metadata supports exam filtering and attribution. A problem carries its own stable identifier and display label, statement, and one or more topic tags.

The parser normalises older topic names (`nghiem-nguyen` to number theory and `phuong-trinh` to algebra) to the five public topics. Books, lessons, and documents are parsed into separate runtime collections. The data model is intentionally source-aware: the collection retains source URL and source name so individual problem pages can display the origin. Some raw problem records also contain a difficulty `level`; the active runtime parser does not retain that field, so difficulty is not currently exposed as a search filter or displayed product feature.

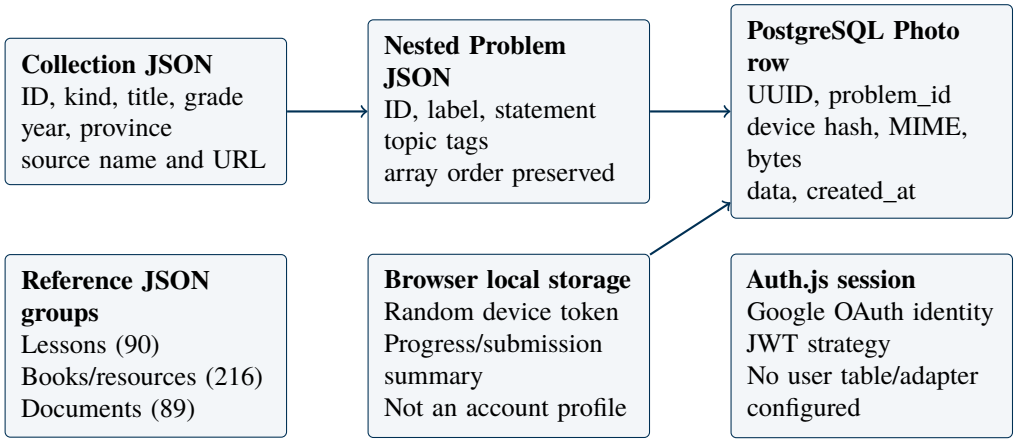


Figure 9. Catalogue and user-related records by storage location. Photo identity and OAuth session remain separate.

The corpus spans five collection kinds: 1,195 HSG, 501 grade-10 specialised entrance, 733 Olympiad, 844 magazine, and 108 topic collections. The current grade metadata contains 237 grade-6, 342 grade-7, 411 grade-8, 1,786 grade-9, and 605 grade-10 collections. The recorded years range from 1935 to 2027; the range is reported as stored metadata, not as an editorial validation of every date. The collection-kind and grade distributions appear in Figures 10 and 11.

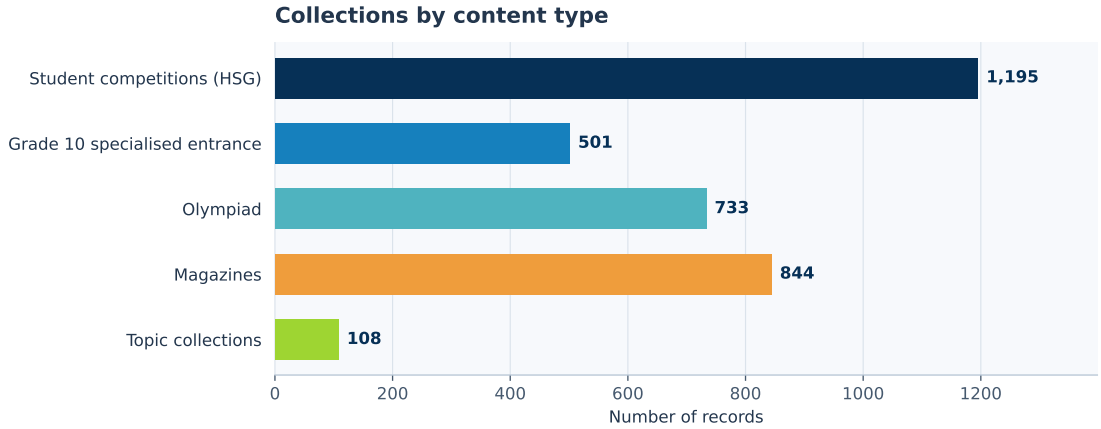


Figure 10. Collection counts by source kind in the inspected JSON corpus.

Topic tags are multi-valued. The current records contain 7,643 number-theory assignments, 12,677 algebra assignments, 3,926 inequality assignments, 10,442 geometry assignments, and 4,098 combinatorics assignments. Since a problem can carry more than one tag, these values are not a partition of the 33,929 problems. Figure 12 visualises these overlapping assignments. The raw corpus has 4,955 problems with level 1, 11,076 with level 2, 8,023 with level 3, 938 with level 4, and 8,937 without a level value. These source fields are incomplete and the active web parser discards them; they are reported as a data-quality observation, not as a difficulty feature available to learners.

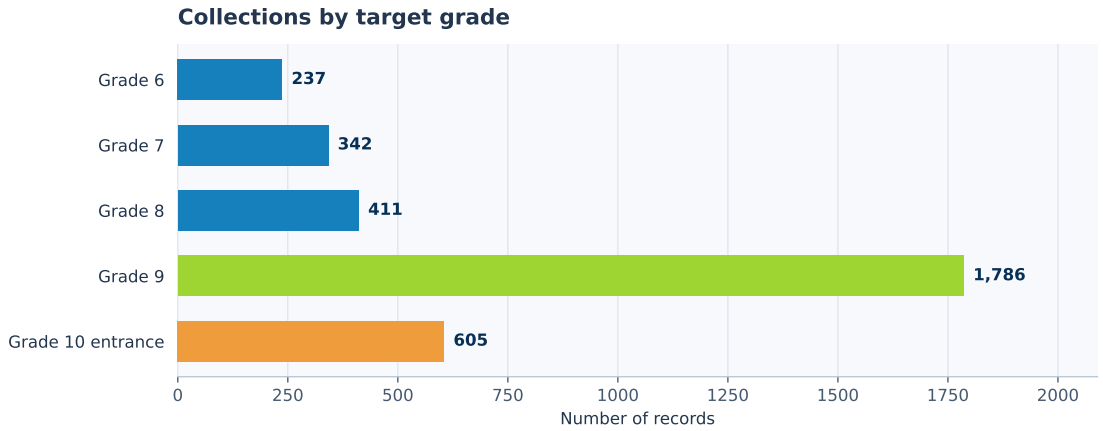


Figure 11. Collection counts by target grade; grade 10 denotes specialised entrance material.

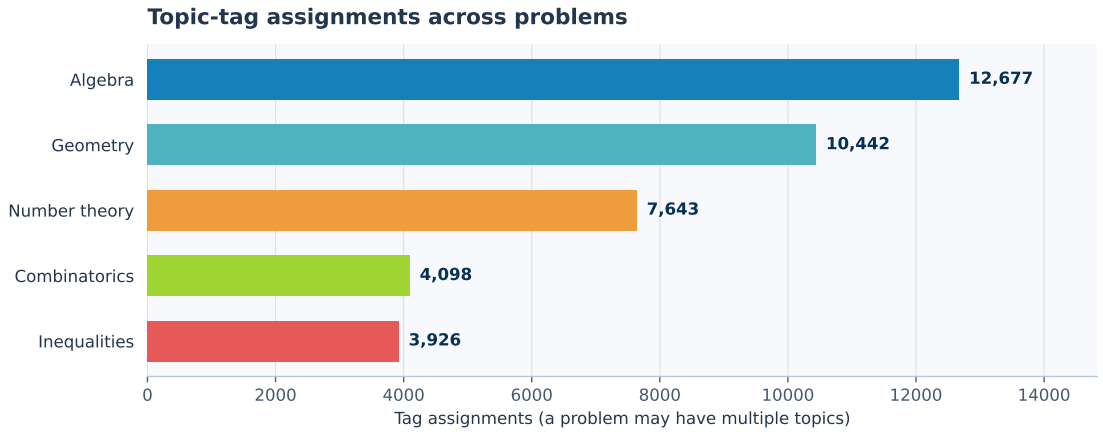


Figure 12. Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars.

2.5. Database design

PostgreSQL is used for mutable photo submissions, not for the static catalogue. The migration creates schema `thcs_hard`, a small `meta` table for schema version, and `photos`. The photo table has a UUID primary key, device hash, problem ID, label, title, MIME type, byte count, binary `bytea` content, and timestamp. One index on `(device, created_at)` supports device history ordered by time; another on `problem_id` supports problem-scoped lookups. The application runs append-only migrations under a PostgreSQL advisory transaction lock when the database module is first used.

There is no foreign key from `photos.problem_id` to JSON content because those problems are not rows in PostgreSQL. The API also does not verify that an uploaded problem ID exists in the JSON catalogue. The current OAuth setup has no Auth.js database adapter or application user table. In the inspected checkout, the session strategy is JWT; the shared layout reads the session to render header controls. The Google identity is not used by the photo table, which remains scoped by a hashed device token.

User data and privacy boundary. The photo feature stores no separate student profile or student-name column. The browser keeps a random UUID-like token and a compact progress summary in local storage. For API requests, the server accepts a token between 20 and 64 alphanumeric/hyphen characters and stores a SHA-256 hash of it on each photo row. The database therefore links photos to a pseudonymous browser device. Clearing browser storage can strand access to that device's history; there is no account recovery or cross-device transfer for those

records. Google OAuth, when configured, provides an Auth.js session but is not connected to photo ownership. Photo contents themselves are user-submitted image bytes and may include personal writing; the application code does not extract or separately index names from those images.

2.6. API design

The custom API is a small read/write surface for photos. Table 3 lists its source-confirmed request and response behaviour. Learning pages read the catalogue through server-side functions rather than a public JSON endpoint. Photo routes use the `x-device` header for device-scoped operations; image retrieval by UUID is the exception.

Method	Path	Request	Successful result / explicit errors
GET	`/api/photos`	`x-device`; optional `?problem=id`	`{photos:[id,pid,label,title,at]}` newest first, max 500; missing token: 401
POST	`/api/photos`	Multipart: `problem`, `label`, `title`, repeated `images`; `x-device`	`{photos:[...]}`; invalid fields/files: 400; daily quota: 429; missing token: 401
GET	`/api/photos/{id}`	UUID in path; no device header required	Image bytes, stored MIME, private immutable one-year cache; malformed or missing UUID: 404
DELETE	`/api/photos/{id}`	UUID in path and matching `x-device`	`{ok:boolean}`; invalid token or UUID: 400
GET/POST	`/api/auth/*`	Auth.js provider and callback protocol	Google OAuth callback and Auth.js session handling in current checkout

Table 3. Implemented route responsibilities and the explicit response behaviour visible in source.

For photo upload, the browser converts the selected image to grayscale, increases contrast, resizes its longest edge to at most 1,000 pixels, and attempts WebP compression with JPEG fallback. The request contains one to six non-empty image files; the server accepts only `image/webp` or `image/jpeg`, rejects files over 150,000 bytes, trims text fields to fixed maximum lengths, and checks the previous 24-hour per-device count against the 60-image limit. Accepted bytes are inserted into PostgreSQL inside a transaction. The quota count occurs before that insert transaction, so simultaneous requests can race and exceed the limit.

Figure 13 shows the upload, retrieval, and deletion lifecycle. The list endpoint returns metadata only and caps results at 500 without pagination. The image endpoint treats the UUID URL as a bearer capability: it does not compare a device token, although UUIDs are difficult to guess. Deletion is stricter: it validates UUID syntax and deletes only a row matching both UUID and hashed device ID. SQL values are passed through the parameterised tagged-template client. No request-size benchmark, concurrent quota test, or upload reliability test was recorded.

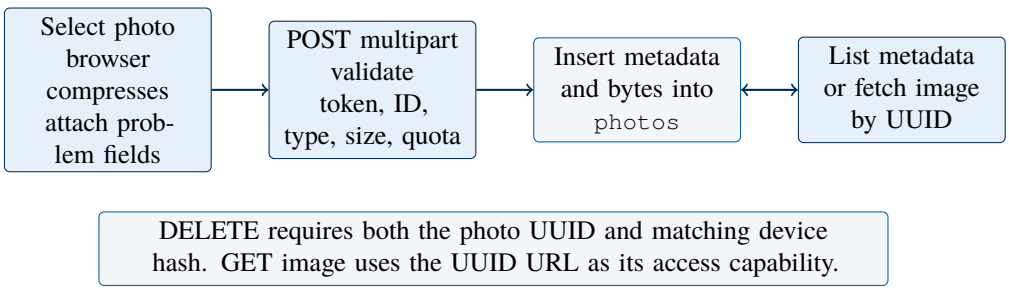


Figure 13. Photo API flow from client-side preparation through database storage and device-scoped deletion.

The current auth source configures Google as the only provider, uses JWT sessions [1], and directs unauthenticated users to `/login`. Sign-in and sign-up are two views of the same Google OAuth operation; first-time provider sign-in is described as account creation in the UI. No email/password credential table is implemented. Auth-related environment variables are documented in `AUTH_SETUP.md`; their values are deliberately excluded from this report.

3. Implementation

3.1. Project structure and runtime responsibilities

The repository follows Next.js App Router conventions. Route files provide page rendering or API handlers; shared UI and library modules keep parsing, topics, rendering, and browser state out of the individual pages.

Listing 1. Relevant Rùa Toán project layout.

```
ruatoan/
|-- app/
|   |-- page.tsx                # home and daily problem
|   |-- luyen/                  # problem search and filters
|   |-- bai/[id]/               # problem detail
|   |-- de/ and de/[id]/        # collection list/detail/print
|   |-- chuyen-de/              # topic pages
|   |-- ly-thuyet/[id]/         # lesson detail
|   |-- sach/                   # books, magazines, resources
|   |-- bai-nop/                # submitted photo history
|   |-- login/, register/       # Google OAuth entry pages
|   |-- api/photos/             # photo API
|   |-- api/auth/[...nextauth]/ # Auth.js handlers
|   `-- ui/                     # shared cards and client components
|-- content/                    # source, lesson, book, and document JSON
|-- lib/                        # content cache, topics, Markdown, database client
|-- scripts/                    # source-specific collection/processing tools
|-- next.config.ts              # tracing and redirects
`-- vercel.json                 # deployment framework and region
```

3.2. Content loading, indexing, and search

At runtime, `lib/content.ts` reads JSON files from `content/`. Parsed file values are cached by file modification time. A content store combines the source files, normalises topic aliases, derives practice classification from titles, and builds maps by collection ID, problem ID, and topic. Duplicate collection IDs keep the version with more problems; duplicate problem IDs are first-wins after collection ordering. Detail lookups therefore use maps, while list pages use in-memory filtering and pagination.

Text search folds Vietnamese diacritics, converts `đ` to `d`, lowercases, and checks that every query token is a substring of the normalised title, source, province, label, or problem statement. The query is capped at 80 characters by the page handler. Problem results show 20 items per page; collection results show 24. Topic maps reduce the candidate set for topic queries, but general text search scans the applicable in-memory list. This is a simple and maintainable design for a read-heavy catalogue, although the project has not benchmarked its memory use or response time at larger corpus sizes.

The homepage selects a daily problem deterministically from the Vietnam calendar date. A stable hash of that date indexes the current problem list, so visitors see the same daily item for a given date and corpus ordering. Related problems are selected deterministically from the same first topic while excluding the current collection.

The repository contains `scripts/web/build-index.mts`, a separate compact-index and prerendering experiment. The current `package.json` does not wire it to a prebuild or predev command, and `next.config.ts` excludes `.data/` from server output. Accordingly, this report describes the active runtime path in `lib/content.ts`; it does not claim that the experiment's generated search index is deployed.

3.3. Statement and mathematics rendering

Problem, lesson, title, and description content is stored as plain text with a deliberately small Markdown-like syntax. `lib/md.ts` escapes ordinary HTML characters and supports paragraphs, one to three heading levels, unordered lists, bold and italic text, simple pipe-delimited tables, and LaTeX expressions. Inline expressions accept dollar or parenthesis delimiters; display expressions accept double-dollar or bracket delimiters. KaTeX [2] renders the mathematical fragments server-side with error throwing disabled, so an invalid expression falls back to escaped text instead of inserting raw HTML. The renderer is intentionally narrower than full Markdown.

3.4. Page composition and interactions

Home. `app/page.tsx` combines current catalogue statistics, daily problem, counts by topic, and six recent collections. It declares a one-hour revalidation.

Practice. `app/luyen/page.tsx` parses validated topic, grade, kind, year, and text filters from query parameters; it displays 20 problem cards per page.

Collections. `app/de/page.tsx` validates kind, exam type, grade, year, province, and text query, then presents 24 collection cards per page. The detail route preserves question order and exposes print and problem-level links.

Problems. `app/bai/[id]/page.tsx` shows collection metadata, topic chips, rendered statement, submission control, previous/next navigation, source, and up to four related problems. Detail pages declare one-day revalidation.

Learning material. Topic pages combine lessons, documents, books, and problem counts; lesson pages render body text and examples and link back to topic practice.

Submissions. The client shrinks images in a canvas before calling the photo API. The history UI lists metadata and loads images only when needed. Browser storage keeps a progress summary separate from stored image rows.

The shared layout uses Be Vietnam Pro for body text and Bungee for display headings. Topic cards and problem tags reuse the five topic colours, while the navigation remains horizontally scrollable at smaller widths and card grids reduce their columns at responsive breakpoints. The layout provides a skip-to-main link, labelled navigation, current-page state, visible keyboard focus, and a reduced-motion preference rule. Print CSS removes navigation, breadcrumbs, action controls, and hidden details so the ordered exam content is the focus. These source-level affordances have not been evaluated in a formal accessibility test.

3.5. Content preparation pipeline

The content directory is the prepared output of source-specific scripts, not a live crawl performed while a page is requested. The script tree contains separate workflows for exam papers, topic material, magazines, olympiad collections, and books/resources. Table 4 maps these workflows to their prepared JSON output.

Exam ingestion scripts reference Gemini model lanes and a Claude processing lane, with cached responses and merge behaviour. This shows that AI-assisted extraction tools exist in the content workflow; it does not prove which tool produced each saved record or that every record received human verification. The project specification asks for source attribution, short attributed magazine statements, metadata-and-link-only book entries, and no copied PDF scans in the served content directory. These are documented project rules, not an audit result for every

Pipeline area	Observed source workflow	Prepared output / control
Exam papers	Crawl source catalogue and PDF links; extract and repair problem statements; assign labels, topics, and metadata	`content/sources/toanmath-*.json`; cached downloads and intermediate results are under ignored `.cache/`
Topic collections	Collect source posts and select material for structured import	`content/sources/toanmath-chuyende.json` and related source files
Magazines	Separate collection/import scripts for magazine sources; retain issue/source context	`content/sources/tapchi-*.json`
Olympiad	Build source specifications, fetch/extract selected papers, and normalise their problems	`content/sources/olympic-*.json`
Lessons	Organise theory records by topic and use lightweight Markdown/KaTeX syntax	`content/theory/*.json`
Books/resources	Crawl or merge bibliographic metadata, links, topics, grades, and rights notes	`content/books.json`
Documents	Maintain topic-oriented metadata and source/PDF links	`content/docs.json`

Table 4. Source preparation responsibilities visible in the scripts and content layout.

record. A continuing editorial review should check source links, dates, duplicates, topic assignment, transcription fidelity, and reuse permissions.

3.6. Google authentication in the current checkout

`auth.ts` configures Google OAuth through Auth.js and selects JWT sessions. The shared layout reads the session to render the appropriate header state; `/login` and `/register` use the same provider flow, and the API catch-all exposes Auth.js GET and POST handlers. The application requests Google consent with account selection, so a user can choose which Google account to use. No database adapter or application user table is configured. A session identity therefore supports the header experience but does not establish persistent application profiles or access control for the photo API. The OAuth files are uncommitted in the audited working tree; production availability remains to be confirmed.

Local or hosted configuration requires `AUTH_SECRET`, `AUTH_GOOGLE_ID`, and `AUTH_GOOGLE_SECRET`; the callback URL must be registered with Google. The project also reads `DATABASE_URL` or `POSTGRES_URL` for PostgreSQL. Secret values are intentionally not included here. The local connection file is ignored by Git.

3.7. Database operations and photo API

The PostgreSQL client [3] is created lazily. On first use, the application checks the schema version and applies missing append-only migrations under an advisory transaction lock. The migration creates `thcs_hard.photos`, its indexes, and a schema metadata table. Image bytes are stored in a `bytea` column, while list queries return only ID, problem context, and time metadata.

The photo client sends its token in `x-device`. The server validates its character shape and hashes it with SHA-256 before using it in queries. The client converts camera images (which can be several megabytes before processing) to grayscale WebP or JPEG with a maximum long edge of 1,000 pixels. The code comments state typical output near 30–80 KB; the route enforces the 150,000-byte ceiling. This is a source-code design description, not a measured upload-size distribution.

The photo route handlers use parameterised tagged SQL. Deletion includes both photo UUID and device hash in its predicate. Image retrieval checks UUID syntax and row existence but does not require the device token; the code treats the UUID URL as the access capability. The API has no pagination for metadata history and no

relational validation that the submitted problem ID exists in the JSON catalogue. The quota count and insert transaction are separate, leaving a race under concurrent requests. These are concrete areas for future hardening rather than measured exploits.

3.8. Deployment and environment

`vercel.json` selects the Next.js framework and the `sin1` region. Vercel's Next.js deployment platform is documented by the provider [5]. The default site URL in layout metadata is `https://ruatoan.vercel.app`; the project can override it with `NEXT_PUBLIC_SITE_URL`. Photo API operation requires a valid PostgreSQL connection string in the deployment environment. OAuth operation separately requires the Auth.js and Google OAuth variables. This report does not identify the production database provider from source comments alone, and it does not reproduce any connection string or credential.

The public demo returned HTTP 200 and the screenshots in Chapter 2 loaded during the report inspection on 1 November 2026. That confirms the site was reachable for those observations, not that all routes, OAuth callbacks, uploads, database writes, or deployment environment variables were validated.

4. Results and Conclusion

4.1. Evidence collected

Table 5 distinguishes source-confirmed facts from observations and checks that were not run. The report review counted non-sample JSON entries directly and inspected the active page, content, database, and route-handler source. It also opened the public demo and captured the home page, practice list, problem page, exam page, topic landing page, and a narrow viewport on 1 November 2026. The demo returned HTTP 200 for the home-page request. This is evidence of availability and visible layout at that time, not proof that all production operations work correctly.

Area	Evidence in this report	Status
Catalogue size	JSON files counted by record group and source	Measured from repository
Route and data logic	Source code inspected for filters, parsing, API validation, and schema	Source-confirmed
Visual presentation	Screenshots captured from public demo routes at a stated date	Observed
Home route availability	Public home page responded HTTP 200 at inspection time	Observed once
Photo submission and retrieval	No end-to-end image write/read/delete test was run	Not tested
OAuth sign-in	Local Auth.js/Google configuration inspected; production callback not tested	Source only
Search quality and response time	No relevance study or performance benchmark collected	Not measured
Accessibility and print output	No formal audit or print comparison collected	Not measured

Table 5. Evidence level for the claims made in this report.

The package manifest provides development, build, and start commands but no declared test command or test runner. The report author did not run a build or test suite while preparing this draft. Therefore, feature descriptions in Chapters 2 and 3 are based on source inspection, not on a pass/fail acceptance run. The OAuth additions in the current checkout are uncommitted, so they must not be represented as confirmed production functionality.

4.2. What the implementation achieves

Rùa Toán combines 3,381 collections and 33,929 problems with lessons, books, magazines, resources, and documents in a single browseable product. The current repository contains 90 lessons, 216 book/resource entries, and 89 document entries. Source-oriented metadata, topic filters, paginated lists, individual problem pages, and full collection views connect the corpus to practical study workflows. KaTeX rendering and print controls address the special handling needed for mathematics content.

The static JSON catalogue and runtime photo store have different responsibilities. The catalogue is easy to version and prepare in bulk; PostgreSQL handles the mutable image records. Browser-side image reduction limits the bytes sent to the photo API. The local progress summary and device token allow basic same-browser continuity without requiring a profile for photo submission. The current checkout also adds a Google OAuth session UI, although it is not yet connected to photo ownership or a persistent app-user table.

4.3. Limitations and risks

L1. **Catalogue validation.** Counts describe raw JSON entries and do not prove uniqueness or correctness. The stored year range includes 1935–2027, and the corpus contains duplicate IDs resolved by runtime rules. Editorial review is needed for dates, duplicates, source links, mathematical transcription, and topic labels.

L2. **Runtime search.** General text queries scan a normalised in-memory list. The current corpus is about 16.6 MB of JSON, but no benchmark establishes how response time or memory will change as it grows. The experimental generated index is not wired into the package's current build path.

L3. **Photo history limits.** Device history is capped at 500 records and has no pagination. The daily quota check is separate from the transaction that inserts the images, so simultaneous requests can pass the same pre-check.

L4. **Photo access model.** Delete requests verify device ownership; image retrieval relies on the UUID URL as a bearer capability and does not verify a device token. Device tokens can be lost when browser storage is cleared and cannot recover across devices.

L5. **User identity separation.** Google OAuth session information is not linked to photos. There is no app-user table, account recovery, or server-side progress record in the inspected code. Auth files are uncommitted and deployment is unknown.

L6. **Quality and operations evidence.** No automated suite, upload test, load test, accessibility audit, search relevance review, or measured Core Web Vitals/Lighthouse result has been recorded.

L7. **Source use.** The content specification sets different rules for exams, short magazine statements, and book metadata, but those rules need record-level auditing and source permission review. Script presence alone does not establish that every catalogue item complies.

4.4. Recommended next steps

- Add a reproducible content validation command for schema, duplicate IDs and statements, broken source links, unexpected years, required attribution, and required metadata coverage.
- Run a focused acceptance pass for filtering, detail navigation, KaTeX examples, print output, photo upload/list/get/delete, limits, and OAuth callbacks; record environment, date, device, and results.
- Measure server memory and response time for representative catalogue filters and text searches before replacing JSON. If needed, build the generated index into the active deploy path or move search to an indexed

database/search service.

- Make daily photo quotas atomic under concurrent requests and add pagination or a cursor to photo history.
- Decide whether image retrieval should remain a UUID capability or require device or account verification; define retention and deletion rules for submitted images.
- If Google sign-in is intended to become an actual account system, add a persistent account model, document the profile fields stored, define privacy/retention rules, and explicitly connect account identity to user data. Otherwise, avoid describing the UI as persistent registration.
- Complete source and permission review for every data provider and keep content attribution visible in the product.

4.5. Conclusion

Rùa Toán has a clear educational purpose and a substantial, source-aware mathematics catalogue. Its architecture separates prepared learning content, browser state, photo submissions, and the newly added OAuth session. The implementation demonstrates practical features for discovering and reading mathematical problems, while its size makes data validation, source review, and performance measurement increasingly important. The next development stage should add evidence through repeatable functional and quality checks and resolve the boundaries between device-scoped photos and account identity before claiming cross-device or account-backed learning records.

References

[1] Auth.js Contributors. Auth.js core reference, 2026. Accessed: 2026-11-01.

[2] KaTeX Contributors. Katex api documentation, 2026. Accessed: 2026-11-01.

[3] PostgreSQL Global Development Group. Postgresql documentation: Data types, 2026. Accessed: 2026-11-01.

[4] Vercel. Next.js app router: Server and client components, 2026. Accessed: 2026-11-01.

[5] Vercel. Next.js on vercel, 2026. Accessed: 2026-11-01.

A. Appendix

A.1. Local setup

From a checkout of the Rùa Toán repository:

```
npm install
npm run dev
# production build and start
npm run build
npm start
```

The development server uses the Next.js default local port unless configured otherwise. The database module reads `DATABASE_URL` or `POSTGRES_URL`; it creates its application schema and photo table on first use if the configured database role has permission. Google sign-in additionally reads `AUTH_SECRET`, `AUTH_GOOGLE_ID`, and `AUTH_GOOGLE_SECRET`. Local setup instructions and Google callback registration are recorded in the source repository's `AUTH_SETUP.md`. Secret values must be supplied through a local ignored environment file or the deployment platform's environment settings and must never be committed.

A.2. Route inventory

Table 6 provides a compact route map for the pages and handlers described in the implementation chapters.

Route	Purpose
`/`	Discovery dashboard and daily problem
`/luyen`	Problem search, filters, topic selection, and pagination
`/bai/{id}`	Individual problem, source, navigation, and photo submission
`/de` and `/de/{id}`	Exam collection search, ordered paper, and print action
`/chuyen-de` and `/chuyen-de/{id}`	Topic catalogue and grouped material
`/ly-thuyet/{id}`	Lesson body, examples, and sibling lessons
`/sach`	Books, magazines, and external resources
`/bai-nop`	Photo submission history for the current browser device
`/login`, `/register`	Google OAuth sign-in and sign-up views in the current checkout
`/api/photos`	Photo metadata listing and multipart upload
`/api/photos/{id}`	UUID image retrieval and device-scoped deletion
`/api/auth/*`	Auth.js provider and callback handlers

Table 6. Application page and API route map.

A.3. Content inventory snapshot

The audited non-sample data files contained the following records on 1 November 2026:

- 11 source JSON files: 3,381 collections and 33,929 problem records;
- 7 theory JSON files: 90 lessons;
- `books.json`: 216 books, magazines, and resource records;
- `docs.json`: 89 document records.

The topic assignment counts and collection breakdown are shown in Figures 10–12. These are repository measurements and should be regenerated if the content files change.

A.4. Data pipeline diagram

Figure 14 summarises how prepared content reaches the runtime pages.

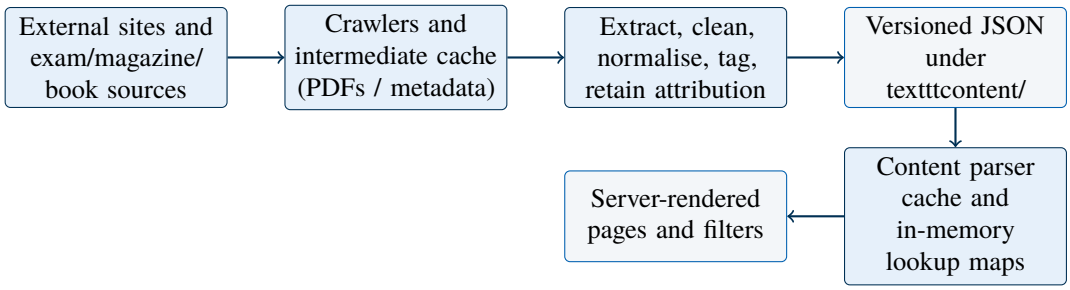


Figure 14. Content preparation and runtime consumption. Source scripts are optional offline tools; they are not called during page requests.

The preparation stage includes AI-assisted extraction helpers in some exam workflows. The diagram shows the stages represented by the scripts, not a claim that every record passes through the same extractor or an equivalent manual review.

A.5. Glossary

Collection An ordered source-aware unit such as an exam paper or topic exercise set.

Problem One individually addressable statement nested in a collection.

Topic tag A classification label; one problem may have multiple tags.

Device token A random value held in browser storage and hashed before database use.

JWT session Auth.js session strategy used by the Google OAuth code in the current checkout; it is separate from photo ownership.

Raw record count Count of JSON objects by record group before claims about uniqueness, correctness, or editorial review.

A.6. AI assistance and academic disclosure

This report draft was prepared with ChatGPT assistance after inspecting the Rùa Toán source and data files. ChatGPT helped organise and describe the implementation; it did not run the application test suite, upload a photo, or

perform an editorial review of the mathematics catalogue. The repository also contains exam-ingestion scripts that call Gemini and a Claude processing lane. The group should disclose any AI tools used during implementation, state which content or code they assisted with, and follow the course's academic-integrity rules.